Solució de l'Examen Parcial EDA 31/10/2025

Proposta de solució al problema 1

(a) Anem a veure com evolucionen els valors de les tres variables a cada iteració. Després de la iteració k-èsima, el valor d'i és k. Com que s va acumulant la suma de totes les i, en acabar la iteració k-èsima s val $1 + 2 + \cdots + k = k(k+1)/2$. Així doncs, després de k iteracions q val $2s - i = 2k(k+1)/2 - k = k^2$.

Si volem saber el nombre exacte d'iteracions que fa el bucle dins f cal trobar l'enter k més petit tal que $k^2 \geq n$, i aquest k és precisament $\lceil \sqrt{n} \rceil$.

Pel que fa al cost asimptòtic d'una crida a $f(n)$ sabem que el pas de paràmetres té cost $\Theta(1)$, perquè només es passa un enter. Totes les operacions dins la funció tenen cost $\Theta(1)$: assignacions, comparacions, i operacions aritmètiques bàsiques entre enters. Com que sabem que el nombre d'iteracions és $\Theta(\sqrt{n})$, i cada iteració te cost $\Theta(1)$, el cost total és $\Theta(\sqrt{n})$.

(b) El cost de l'algorisme A ve donat per la recurrència $A(n) = 3 \cdot A(n/3) + \Theta(n)$. Si apliquem el teorema mestre per a recurrències divisores de l'estil $T(n) = a \cdot T(n/b) + \Theta(n^k)$ podem identificar que $a = b = 3$, $k = 1$, i $\alpha = \log_b(a) = \log_3(3) = 1$. Com que $\alpha = k$, podem concloure que $A(n) \in \Theta(n \log n)$.

Pel que fa a l'algorisme B, la recurrència que descriu el seu cost és $B(n) = B(n-1) + \Theta(n^p)$. Si apliquem el teorema mestre per a recurrències substractores de l'estil $T(n) = aT(n-1) + \Theta(n^k)$ podem identificar $a = c = 1$, i $k = p$. Com que $a = 1$, el seu cost és $\Theta(n^{p+1})$.

Per tant, és fàcil determinar que:

- Si $p = 0$, aleshores A és asimptòticament pitjor que B perquè $n \log n \in \Omega(n)$ però $n \log n \notin O(n)$.
- Si $p \geq 1$, aleshores A és asimptòticament millor que B, perquè $n \log n \in O(n^{p+1})$ però $n \log n \notin \Omega(n^{p+1})$.

(c) Considerem un node n d'un ABC tal que té subarbre dret no buit. Si escrivim totes les claus de l'ABC amb un recorregut en inordre de l'ABC, existirà un segment del resultat amb la forma:

inordre(subarbre esquerre d'n)	n	inordre(subarbre dret n)

Per tant, l'element que s'escriurà immediatament després d'n (el seu successor) estarà en el subarbre dret d'n. A més, com que l'inordre escriurà tots els elements del subarbre dret de petit a gran, el successor serà precisament l'element més petit del subarbre dret. Aquest element es pot obtenir partint de l'arrel del subarbre dret i movent-nos sempre cap al fill esquerre fins a trobar un node que no en tingui. Aquest serà el successor.

Nota: no és suficient dir que com que els elements del fill esquerre d'n són més petits i els del fill dret són més grans, el successor estarà al fill gran. Això no és suficient perquè poden existir claus més grans que la d'n que no estan en el fill dret (pujant cap al pare i movent-nos a la dreta, per exemple).

Proposta de solució al problema 2

(a)
```
int cota_superior ( ) {
  int p = 1;
  while (menor_igual_que_n(p)) p *= 2;
  return p;
}
```

El nombre sup ens servirà per poder buscar n dins l'interval $[1, sup)$. Ho farem amb una cerca binària amb crida inicial $nombre(1, sup)$:

```
int nombre (int l, int r) {
  if (r == l+1) return l;
  else {
    int m = (l+r)/2;
    if (menor_igual_que_n(m)) return nombre(m,r);
    else return nombre(l,m);
  }
}
```

Per tal de justificar el cost, considerem primer el càlcul de sup amb un cost $\Theta(\log n)$. A continuació cal analitzar el cost de la crida $nombre(1, sup)$. El paràmetre que decreix a cada crida recursiva és la diferència entre l i r, que anomenarem s. Podem veure que la funció *nombre*, a part de la crida recursiva, només fa operacions de cost $\Theta(1)$. Així doncs, el seu cost ve descrit per la recurrència $C(s) = C(s/2) + \Theta(1)$, que té solució $\Theta(\log s)$. Com que en la crida inicial $s = sup - 1 + 1 = sup$, i sabem que $sup \leq 2n$, podem garantir que el cost és com a molt $O(\log 2n) = O(n)$.

Fem notar que si calculem sup de la manera que hem presentat, aleshores la crida inicial pot ser millorada: $nombre(sup/2, sup)$. Això ho podem garantir perquè sabem que $sup/2$ és menor o igual que n. L'interval de la crida inicial te ara mida $sup/2 + 1 \leq n + 1$, pel que aconseguim el mateix cost asimptòtic $O(\log n)$.

(b) La idea és que podem escriure $S_n = S_{n-1} + F_n$. Per tant, podem escollir S_i com a S_{n-1} i prendre $A = B = 1$, $C = 0$ i $F_j = F_n$. Per a obtenir F_{n+1}, com que sabem que $F_{n+1} = F_n + F_{n-1}$, podem prendre $D = 0$, $E = F = 1$ i $F_k = F_{n-1}$. Finalment, per a obtenir F_n podem escollir $G = 0$, $H = 1$, $K = 0$. Així doncs, tenim la recurrència matricial:

$$\begin{pmatrix} S_n \\ F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} S_{n-1} \\ F_n \\ F_{n-1} \end{pmatrix}$$

Com que $S_0 = F_0 = F_1 = 1$, aleshores es compleix que

$$\begin{pmatrix} S_n \\ F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \end{pmatrix}^n \cdot \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

Per tal de calcular S_n em temps $\Theta(\log n)$ calcularem la n-èsima potència de la matriu 3×3 en temps $\Theta(\log n)$ utilitzant l'exponenciació ràpida, i a continuació la multiplicarem pel vector columna $(1, 1, 1)$, en temps $\Theta(1)$ perquè la mida no depèn d'n. El primer element del vector resultant és S_n.

Una altra possible solució que considerarem correcta és adonar-se que es compleix $S_n = F_{n+2} - 1$, demostrar-ho per inducció i esmentar que només ens cal calcular F_{n+2} i restar-hi 1. Això es pot fer en temps $O(\log n)$ utilitzant l'exponenciació ràpida sobre la matriu M de mida 2×2 que defineix la recurrència matricial de Fibonacci per calcular M^{n+2}, multiplicar M^{n+2} pel vector columna $(1, 1)$ i adonar-se que F_{n+2} és el primer element del vector resultant.